



Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

SWE 4802: Software Maintenance Lab

Summer, 2024-2025

Lab 5

Code Clone Detection and Analysis

Namisa Najah Raisa

210042112

Program: B.Sc in SWE

Department of CSE

Date: August 16, 2026

Contents

1	Project Link	2
2	Rsearch code clone detection tools	2
2.1	Description	2
2.2	Why I selected jscpd	2
3	Methodology	2
4	Results	3
4.1	Overall Statistics	3
5	Analysis	4
5.1	Locations	4
5.2	Clone Types	6
5.3	Impact on Maintainability	6
6	Proposed Solutions	7

1 Project Link

Github Link of the project- <https://github.com/N4M154/The-Opinionated>

2 Rsearch code clone detection tools

2.1 Description

I selected the tool **jscpd** (JavaScript Copy/Paste Detector), an open-source, token-based clone detector distributed via npm. *jscpd* supports more than 150 languages and formats. It tokenizes source files and applies a Rabin–Karp string-matching algorithm over the token stream to locate duplicated fragments whose length exceeds a configurable threshold (`--min-tokens`). Because matching is performed on tokens rather than raw text, jscpd detects both *Type 1 clones* (exact copies modulo whitespace and comments) and *Type 2 clones* (copies with renamed identifiers or changed literal values). It reports each clone pair with precise file paths and line ranges, per-file and project-wide duplication percentages, and can emit console, JSON, and interactive HTML reports.

2.2 Why I selected jscpd

Several established clone detectors were considered before selecting jscpd:

- **PMD CPD** is a mature token-based detector, but it is Java-centric in its tooling, requires a Java runtime and manual distribution setup, and its JavaScript/JSX support is weaker than its Java support.
- **Simian** is fast and multi-language but is commercial/closed-source and produces text-only reports without visualization.
- **NiCad** detects Type 3 (near-miss) clones with high accuracy but requires a Linux/TXL toolchain, per-language grammars, and non-trivial configuration - disproportionate effort for a single-project study.
- **SourcererCC** scales to very large corpora but is research-oriented software with a multi-step indexing pipeline unsuited to quick, single-repository analysis.

I chose jscpd because it best fits this project's characteristics: (i) *first-class JavaScript/JSX support*, matching the MERN codebase; (ii) *zero-installation execution* via `npm`, with no runtime dependencies beyond Node.js, which the project already requires; (iii) *built-in HTML reporting* that shows clone pairs side by side, simplifying manual inspection and evidence collection; (iv) *fine-grained scoping* through glob-based `--ignore` patterns, essential for excluding generated files; and (v) explicit classification-relevant output covering Type 1 and Type 2 clones.

3 Methodology

The analysis followed these steps:

1. **Scoping.** The analysis was iteratively scoped so that it reflects only developer-written source code. In an initial unscoped run, the vast majority of reported duplication originated from `package-lock.json`, an npm-generated dependency lock file whose thou-

sands of structurally identical entries are not hand-written code. A second run revealed a further contamination source: the tool's own previously generated HTML report, stored inside the repository, was itself scanned and reported as heavily duplicated markup. Consequently, the final configuration (i) restricts detection to the languages authored in this project (JavaScript and JSX) via the `--format` option, and (ii) explicitly excludes `node_modules/`, `package.json`, `package-lock.json`, the report output directory, and build artifacts.

2. **Tool execution.** After deleting the stale report directory, jscpd was executed from the repository root:

```
npx jscpd . --format "javascript,jsx"
      --reporters html,console --output ./clone-report
      --min-tokens 30
      --ignore "**/node_modules/**,**/clone-report/**,
              **/package-lock.json,**/package.json"
```

The threshold of 30 tokens balances sensitivity and noise: it is small enough to catch duplicated logic blocks yet large enough to suppress trivial matches such as import lists.

3. **Report inspection.** The generated HTML report (`clone-report/html/index.html`) was inspected to record overall statistics, per-file duplication, and individual clone pairs. Representative clones were then examined manually in the source code to classify them and assess their maintainability impact.

4 Results

4.1 Overall Statistics

Image 1 and Image 2 summarizes the project-wide results reported by jscpd.

Format	Files analyzed	Total lines	Total tokens	Clones found	Duplicated lines	Duplicated tokens
css	3	170	776	0	0 (0.00%)	0 (0.00%)
javascript	17	734	3368	2	18 (2.45%)	72 (2.14%)
jsx	24	3075	12016	16	224 (7.28%)	967 (8.05%)
markdown	1	8	59	0	0 (0.00%)	0 (0.00%)
markup	2	26	251	0	0 (0.00%)	0 (0.00%)
Total:	47	4013	16470	18	242 (6.03%)	1039 (6.31%)

Found 18 clones.
HTML report saved to ./clone-report\jscpd-report.html

Figure 1: Terminal



Figure 2: Dashboard

5 Analysis

5.1 Locations

Server-side duplication is concentrated in `api/controllers/review.controller.js`, where the `findById + 404-check` block is repeated three times (lines 87–97, 127–134, 189–199; Figure 3). Client-side duplication accounts for most clone pairs: four helper functions are duplicated between `CarouselCard.jsx` and `ReviewCard.jsx` (Figure 4), `SignUp.jsx` is a near line-for-line copy of `SignIn.jsx`, and headers, SVG icons, and `fetch` boilerplate are pasted across several pages.

javascript

api\controllers\review.controller.js (Line 87:30 - Line 97:6), api\controllers\review.controller.js (Line 189:31 - Line 199:6)

Show code

```
const { id } = req.params;

const review = await Review.findById(id);
// .populate("author", "email");

if (!review) {
  return res.status(404).json({
    success: false,
    message: "Review not found",
  });
}
```

api\controllers\review.controller.js (Line 89:44 - Line 97:6), api\controllers\review.controller.js (Line 127:50 - Line 134:6)

Show code

```
const review = await Review.findById(id);
// .populate("author", "email");

if (!review) {
  return res.status(404).json({
    success: false,
    message: "Review not found",
  });
}
```

Figure 3: Cloned `findById/404` block repeated within `review.controller.js`.

jsx

client\src\components\CarouselCard.jsx (Line 4:3 - Line 40:11), client\src\components\ReviewCard.jsx (Line 52:3 - Line 88:11)

Show code

```

const getReactionCount = (type) => {
  return review.reactions?.filter((r) => r.reactionType === type).length || 0;
};

const formatDate = (dateString) => {
  return new Date(dateString).toLocaleDateString("en-US", {
    year: "numeric",
    month: "short",
    day: "numeric",
  });
};

const getCategoryColor = (category) => {
  const colors = {
    anime:
      "bg-pink-500/20 text-pink-500 border-pink-500/30 dark:text-pink-300",
    movie:
      "bg-blue-500/20 text-blue-500 border-blue-500/30 dark:text-blue-300",
    "tv show":
      "bg-green-500/20 text-green-500 border-green-500/30 dark:text-green-300",
    music:
      "bg-purple-500/20 text-purple-500 border-purple-500/30 dark:text-purple-300",
    book: "bg-yellow-500/20 text-yellow-500 border-yellow-500/30 dark:text-yellow-300",
  };
  return (
    colors[category] || "bg-gray-500/20 text-gray-300 border-gray-500/30"
  );
};

const getReactionColor = (type) => {
  const colors = {
    funny:
      "bg-green-500/20 text-green-500 dark:text-green-200 border-green-500/30",
    creative:
      "bg-yellow-500/20 text-yellow-500 dark:text-yellow-200 border-yellow-500/30",
    love: "bg-pink-500/20 text-pink-500 dark:text-pink-200 border-pink-500/30",
  };
  return (
    colors[type] || "bg-gray-500/20 text-gray-300 border-gray-500/30"
  );
};

```

Figure 4: Helper functions duplicated between CarouselCard.jsx and ReviewCard.jsx.

5.2 Clone Types

jscpd matches token sequences but does not label clone types, so the findings were classified manually.

Type 1 (exact): the vast majority - the controller's triple lookup/404 block, the copied header and SVG icon, the repeated `fetch` calls, and the whole-page `SignIn/SignUp` duplication.

Type 2 (renamed): the card helper functions and a repeated `<select>` wrapper in `ReviewForm.jsx`, identical in structure with only field names/literals differing. No Type 3/4 clones were reported, as these fall outside jscpd's token-based detection.

5.3 Impact on Maintainability

Every fix or change to a cloned fragment must be manually mirrored at each copy - e.g., the commented-out `.populate()` line must be kept consistent across three endpoints, and adding a new category or reaction requires synchronized edits to the color maps in both card components. Independent evolution of `SignIn/SignUp` risks visibly inconsistent authentication flows. At 7.28% duplicated JSX lines and 2.45% in JavaScript, duplication is localized rather than systemic, but it sits in the most change-prone modules (auth pages, review CRUD, review cards), amplifying the practical maintenance risk.

6 Proposed Solutions

The following refactorings target each detected clone group directly.

1. **review.controller.js** - extract the lookup-and-404 pattern into a single helper and call it from `getReviewById`, `updateReview`, and `deleteReview`:

```
async function findReviewOr404(id, res) {
  const review = await Review.findById(id);
  if (!review) {
    res.status(404).json({ success: false,
                           message: "Review not found" });
    return null;
  }
  return review;
}
```

2. **CarouselCard.jsx/ReviewCard.jsx** - move `getReactionCount`, `formatDate`, `getCategoryColor` and `getReactionColor` into `client/src/utils/reviewHelpers.js` and import them in both components.
3. **Header.jsx / SignUp.jsx** - extract the repeated inline SVG into a small `<UserIcon />` component.
4. **Landing.jsx / SignIn.jsx / SignUp.jsx** - extract the shared `<header>` block into a single `<SiteHeader />` component used by all three pages.
5. **ReviewForm.jsx/SignIn.jsx/SignUp.jsx** - wrap the repeated `fetch-POST` pattern in shared API helpers, e.g. `createReview(formData)` in `client/src/api/reviewApi.js` and `signIn/signUp` in `client/src/api/authApi.js`.
6. **ReviewCarousel.jsx (self-clone)** - replace the hand-duplicated array literal with a single constant or a generator function.
7. **ReviewForm.jsx, lines 143–164 (self-clone)** - extract the `select-field-with-gradient-overlay` markup into a reusable `<FormSelectField label options ... />` component instantiated per field.
8. **SignIn.jsx / SignUp.jsx (highest impact)** - these files are approximately 90% identical. Both pages should render a single `<AuthForm mode="signin" | "signup" />` component (or share an `<AuthLayout>` plus a `useAuthForm(mode)` hook) parameterized by a `mode` prop, rather than maintaining two near-identical copies. This refactoring alone would eliminate the majority of the reported JSX duplication.

Adding `jscpd` to the development workflow as an npm script or CI check with a duplication threshold would prevent similar clones from being reintroduced.